RDS 203: Advanced Computational Methods for Disease Modelling

Problem Set 1: Numerical Methods

Submit one consolidated document with your answers (and plots, as appropriate) to all questions below.

Also submit a self-contained file of the Java code developed for each problem (i.e., 10 Java files in total). It is ok if the version of code submitted produces results for the last part of each question, but all code used to generate results for earlier parts of the questions must also be included, and it should be clear how the answers to earlier parts can be re-produced (e.g., by changing parameter values or un-commenting small sections of code).

You may work with others or consult other resources, but each student must submit their own answers and code implementations.

The problem set is due at the end of Module I, but it is recommended that you work on problems as the relevant material is covered in class (see Syllabus or Canvas).

1. Potential Impact Fraction

The potential impact fraction (PIF) is a measure of the impact that a risk factor has on a disease (e.g., incidence). It can be used to estimate the change in disease risk for a given change in a risk factor (e.g., the impact of an intervention).

The PIF can be calculated as:

$$1 - \frac{\int \mathrm{RR}(x) f^*(x) dx}{\int \mathrm{RR}(x) f(x) dx}$$

where $\mathrm{RR}(x)$ is the relative risk at exposure level x, $f(x)$ is the distribution of exposure at baseline, and $f^*(x)$ is the counterfactual distribution of exposure (e.g., after intervention).

For this problem, we'll model the impact of changes in the population body mass index (BMI) on the incidence of 2 diseases: colorectal cancer and stroke (cerebrovascular accident [CVA]).

Note: This is a simplified version of this analysis: Basto-Abreu A, Barrientos-Gutiérrez T, Vidaña-Pérez D, et al. Cost-Effectiveness Of The Sugar-Sweetened Beverage Excise Tax In Mexico. *Health Aff (Millwood)* 2019; 38(11): 1824-1831. PMID: 31682510.

a. Write a function `calcBMI_Density(double x, double mean, double sd)` to calculate the density of a BMI distribution at x for a given mean and standard deviation (SD). We will assume that BMI follows a log-normal distribution. Given a mean M and SD S the log-normal parameters μ and σ are calculated as:

$$\mu = \log\left(\frac{M^2}{\sqrt{S^2 + M^2}}\right)$$

$$\sigma = \sqrt{\log\left(1 + \frac{S^2}{M^2}\right)}$$

The PDF of the log-normal distribution is then calculated as:

$$f(x) = \frac{1}{x\sigma\sqrt{2\pi}} \exp\left[-\frac{(\log(x) - \mu)^2}{2\sigma^2}\right]$$

b. Write a function `calcBMI_RR(double x, double rr)` to calculate the relative risk (RR) for a disease at a BMI of x and a specified relative risk increase per BMI unit over 21 (assumed to be the 'optimal' BMI for this problem):

$$\mathrm{RR}(x) = \max\left(\mathrm{RR}^{(x-21)}, 1\right)$$

For simplicity, values below 21 are included in the reference BMI category.

c. Write a function `integrate(double rr, double mean, double sd)` that calculates $\int \mathrm{RR}(x)f(x)dx$. Use a minimum of $x = 10$ and maximum of $x = 100$ when integrating over the BMI distribution. Use Simpson's Rule and a step size of 0.1 to perform the numerical integration.

d. Write a function `calcPIF(double mean, double sd, double rr, double effectSize)` that calculates the PIF for a given (baseline) BMI distribution, disease RR, and intervention effect size (i.e., reduction in mean BMI, assume the SD remains the same).

e. Write a function `calcCasesAverted(int cohortSize, double baselineInc, double mean, double sd, double rr, double effectSize)` that calculates the cases of disease averted (after 1 year, ignoring competing risks) for a specific age group given the cohort size, baseline disease incidence, BMI distribution (mean and SD) and RR, and intervention effect size.

f. Assume an intervention results in a 1.0 unit reduction of mean BMI (for all age groups). Calculate the number of cases of colorectal cancer and CVA averted (after 1 year) for each of the age groups listed below, and for the population in total.

				BMI RR (per unit)		Baseline Incidence	
Age Group	**Cohort Size**	**Mean BMI**	**BMI SD**	**Colorectal Cancer**	**CVA**	**Colorectal Cancer**	**CVA**
35-39	4,567,000	29.43	5.73	1.03	1.14	0.00002	0.00073
40-44	4,214,000	29.90	5.51	1.03	1.14	0.00004	0.00084
45-49	3,690,000	30.28	5.27	1.03	1.10	0.00008	0.00087
50-54	3,092,000	30.70	5.47	1.03	1.10	0.00013	0.00107
55-59	2,534,000	30.30	4.91	1.03	1.10	0.00018	0.00125
60-64	1,979,000	29.79	5.19	1.03	1.08	0.00025	0.00149
65-69	1,493,000	29.31	4.90	1.03	1.08	0.00032	0.00164
70-74	1,112,000	28.82	5.04	1.03	1.05	0.00042	0.00236
75-79	791,000	27.62	5.17	1.03	1.05	0.00055	0.00378
80-100	1,010,000	26.21	5.52	1.03	1.03	0.00116	0.01073

2. Compartmental Models (SIR)

An SIR model is one of the simplest compartmental models where the population is divided into **S**usceptible, **I**nfectious, and **R**ecovered (or 'removed') compartments. The dynamics for a closed SIR system (e.g., no births or migration) are expressed by the following system of ordinary differential equations:

$$\frac{dS}{dt} = -\beta S \frac{I}{N}, \frac{dI}{dt} = \beta S \frac{I}{N} - \gamma I, \frac{dR}{dt} = \gamma I$$

where β is the infection rate (per contact with an infectious individual), and γ is the recovery rate.

Assume $\beta = 0.3,\ \gamma = 0.1$, and that the total population size $N = 10{,}000$, with everyone starting ($t = 0$) in the **S**usceptible state, except for 1 index **I**nfectious individual: $S_0 = 9{,}999$, $I_0 = 1$.

a. Define functions `newCases(double beta, double S, double I, double N)` and `recoveredCases(double gamma, double I)` to calculate the rate of change in new and recovered cases: $\beta S \frac{I}{N}$ and γI.

b. Define a function `euler(double beta, double gamma, double S, double I, double R, double N, double h)` that calculates forward Euler for the SIR model with a step size of h. This function should return an array with the updated values of S, I, R at $t + h$.

c. Run the SIR model using forward Euler with $h = 1.0$ until $t = 200$. Record the prevalence of each compartment at each (integer) time t and write these to a .csv file (or print them to the console). Create a plot of the epidemic curves (e.g., in R or Excel). *Hint*: Define a cycle counter z, calculate $t = z * h$, and only output values when t is an integer (`t%1==0`).

d. At what (integer) time t^* does the peak of the epidemic occur (i.e., largest number of people in the **I**nfectious state), and what is the value of I at the peak?

e. How does your answer to part d) change using step sizes $h = 0.1$ and $h = 0.01$?

f. Now define a function `rungeKutta(double beta, double gamma, double S, double I, double R, double N, double h)` that uses Runge-Kutta (RK4) for the SIR model with a step size of h. This function should return an array with the updated values of S, I, R at $t + h$. Repeat part d) using Runge-Kutta with step sizes of $h = 1.0$ and $h = 0.1$.

g. Now suppose that people adapt to the epidemic over time. Assume that people vary their contact rate with others based on the number of current infections: $\beta_t = 0.3 - 0.01 * \max(\log(I_t), 0)$. Repeat part d) using Runge-Kutta with a step size of $h = 0.1$.

3. Cubic Spline Interpolation

Suppose we have the following set of points that we want to interpolate using natural cubic splines (i.e., boundary condition $f''(x) = 0$):

x	y
0	0
5	3
7	4
13	9
19	14
25	11
32	5

a. Write code to construct a linear system to calculate the spline coefficients.

b. Solve the system and plot the cubic splines (with step size of 0.1) and original points. Also report the coefficients for each spline.

Hint: Construct a matrix `double matrix[][]` for the left-hand side and an array `double constraints[]` for the right-hand side. You may then use the following functions from Apache Commons Math to solve the linear system:

```
RealMatrix A = MatrixUtils.createRealMatrix(matrix);
DecompositionSolver solver = new LUDecomposition(A).getSolver();
RealVector b=new ArrayRealVector(constraints);
RealVector solution=solver.solve(b);
```

4. Linear Regression

Download the dataset "data_NHANES.csv" from Canvas, which has data for 5,821 individuals from the National Health and Examination Nutrition Survey (NHANES) 2017-2018. We are going to estimate a linear regression model to predict systolic blood pressure based on age, sex, BMI, and waist circumference.

a. Write code to read the dataset into a matrix (`double[][]`) of predictor variables and an array (`double[]`) of the outcome variable. *Hint*: Add a column of 1s to the matrix of predictor variables to estimate an intercept.

b. The closed-form solution for Ordinary Least Squares is given by:

$$\widehat{\boldsymbol{\beta}} = \left(\boldsymbol{X}^{\mathrm{T}}\boldsymbol{X}\right)^{-1}\boldsymbol{X}^{\mathrm{T}}\boldsymbol{y}$$

Estimate $\widehat{\boldsymbol{\beta}}$ using this "normal equation" which involves matrix inversion. You may use `MatrixUtils.inverse()` and other matrix functions from Apache Commons Math.

c. Now estimate the regression coefficients using QR decomposition. You may use `QRDecomposition` from Apache Commons Math. Once you have constructed the equation $\boldsymbol{R}\widehat{\boldsymbol{\beta}} = \boldsymbol{Q}^{\mathrm{T}}\boldsymbol{y}$ calculate $\widehat{\boldsymbol{\beta}}$ using backward substitution.

Coefficients:
beta sex; 93.74776396992581
beta age; -2.485922377919559
beta bmi; 0.4764598219015764
beta waist; 0.32564256809016845
beta meanBP_sys; 0.020958056419428873

5. Cholesky Decomposition

a. Write a function `cholesky(double A[][])` that performs a Cholesky Decomposition on a matrix $\boldsymbol{A}$ using the Cholesky-Banachiewicz algorithm.

b. Calculate the Cholesky Decomposition of the following matrix:

$$\boldsymbol{A} = \begin{bmatrix} 5 & 3 & 1 \\ 3 & 2 & 1 \\ 1 & 1 & 2 \end{bmatrix}$$

L:

2.23607	0.00000	0.00000
1.34164	0.44721	0.00000
0.44721	0.89443	1.00000

c. Calculate the Cholesky Decomposition of the following matrix (this is a rounded version of the variance-covariance matrix from problem 4):

$$\boldsymbol{A} = \begin{bmatrix} 1.95716 & -0.37336 & 0.00484 & 0.05997 & -0.03411 \\ -0.37336 & 0.17668 & -0.00082 & -0.01179 & 0.00497 \\ 0.00484 & -0.00082 & 0.00012 & 0.00037 & -0.00020 \\ 0.05997 & -0.01179 & 0.00037 & 0.00626 & -0.00245 \\ -0.03411 & 0.00497 & -0.00020 & -0.00245 & 0.00109 \end{bmatrix}$$

L:

1.39899	0.00000	0.00000	0.00000	0.00000
-0.26688	0.32474	0.00000	0.00000	0.00000
0.00346	0.00032	0.01039	0.00000	0.00000
0.04287	-0.00108	0.02137	0.06296	0.00000
-0.02438	-0.00473	-0.01099	-0.01866	0.00202

6. Gradient Descent

Download the file "data_covid-MA_2020.csv" which has counts of confirmed and probable cases of COVID-19 in Massachusetts for March-June 2020. You will adapt the SIR model that you implemented for problem 2 and find parameter values that allow you to fit the empirical data.

For this problem we will assume that people vary their contact rate with others based on the number of current infections: $\beta_t = \beta + \alpha * \max\left(\log(I_t), 0\right)$.

We will also adapt the model to account for under-reporting of cases. Assume that the proportion of reported cases varies over time: $\pi_t = \frac{1}{1+\exp[-(\theta t-2)]}$.

You will use gradient descent to find values for the parameters $\beta, \gamma, \alpha, \theta$.

a. Create a function `runSIR(double beta, double gamma, double alpha, double theta)` that runs the SIR model for given values of the parameters. Define N as 6,894,000 (the population of Massachusetts in 2020), $I_0 = 1$, and $S_0 = N - I_0$. The model should run until $t = 150$ using Runge-Kutta (step size of $h = 1$), and the function should return a `double` array of reported cases each day. We'll assume all active cases are Infectious, so return the size of I adjusted for under-reporting: $[\pi_0 I_0, \pi_1 I_1, \ldots \pi_{150} I_{150}]$. Record the number of cases at the beginning of each cycle. (You can modify the function so that it does not print/write any output.)

b. Create a function `writeOutput(double cases[])` that writes the number of daily cases from one run of the model. Run the model with $\beta = 0.6, \gamma = 0.4, \alpha = 0, \theta = 0$ and plot the output compared to the empirical data.

c. Read in the empirical data of case counts to a matrix called `targets` where the first column is t and the second column is the number of cases. Write a function `scoreModel(double targets[][], cases[])` that calculates the loss score for the current model parameters. Define the loss score as the mean squared-error. Re-scale the loss score by dividing it by 1,000,000 so that the gradient is not too large. What is the score for the initial parameter values?

d. Write a function `calcGradient(double beta, double gamma, double alpha, double theta, double targets[][])` that calculates the gradient using finite differences (use the central-difference approach with a step size $h = 0.001$). The function should return an array (of size 4) with the estimated gradient for the current parameter values. What is the gradient for the current parameter values?

e. Estimate parameter values that minimize the loss score using gradient descent with a step size of `h=1e-7`. (You'll want to print the loss score, parameter values, and gradient to the console each iteration so you can track progress/debug.) Repeat the gradient descent loop until the loss score converges: here you can terminate the loop when the score changes by less than `1e-6`. Report the final loss score and parameter values. Plot the model predictions compared to the empirical data.

7. Simulated Annealing – Polynomial

Download the file "data_Polynomial.csv" from Canvas which has 100 noisy samples from a polynomial function of the form: $f(x) = ax^3 + bx^2 + cx + d$. Use simulated annealing (use the "greedy" version) to find values of a, b, c, d that minimize the mean squared-error. Use a standard normal ($N(0,1)$) as the prior probability distribution (i.e., initial sampling distribution) for each parameter. (Use the `nextGaussian()` function from MersenneTwisterFast.) Run the search chain for 10,000 iterations and use an exponential cooling schedule: 0.999^t. Report the fitted parameter values and plot the predicted function compared to the empirical data.

8. Simulated Annealing – Travelling Salesman

Download the file "data_TSP.csv" from Canvas which has the x, y coordinates of 50 cities. Find the shortest path that passes through each city exactly once and returns to the first city (i.e., the "travelling salesman problem").

Create an array `int order[]` that defines a route (i.e., the order that the cities will be visited). The array should be of size 51, and initialized with a random order of cities (sequentially is fine), with the first and last elements set to `0` to indicate a return to the origin city. In each step, propose a new route that is 'close to' the current route. An easy way to do this is to propose a swap of 2 random cities in the route. *Hint*: Use the `nextInt()` function from MersenneTwisterFast to sample random indices to swap. (Make sure to not swap the first or last city in the route, as they should always be `0`.)

Calculate the distance of the proposed route (Euclidean distance between all cities on the route), and accept the swap if the distance is shorter, or with probability $p = \exp\left(-\frac{\Delta}{t}\right)$, where Δ is the difference in distance between the current and proposed route, and t is the current temperature.

Run a search chain using simulated annealing over 100,000 cycles with a starting temperature of 1000 and exponential cooling schedule (`temp*=0.99985`).

Write out the final order of the cities, report the route distance, and plot the route.

9. Sampling

Plot a histogram/density of your sampled values for each of the distributions below.

a. Draw 1000 random samples from the following empirical distribution:

Country	**#**
Nigeria	228
Ethiopia	129
Egypt	115
DRC	106
Tanzania	67
South Africa	63
Kenya	55
Sudan	50
Uganda	49
Algeria	46

b. Draw 1000 random samples from a Binomial distribution: $X \sim \mathrm{Bin}(1000000, 0.01)$. *Hint*: Use `Gamma.LogGamma()` from Apache Commons Math.

c. Draw 1000 random samples from a Beta distribution: $X \sim \mathrm{Beta}(9, 3)$.

d. Draw 1000 random samples from a function with the following density: $f(x) = 5\cos(x^2) - 2x + 25$ for x in $[-10,10]$.

e. Sample 1000 pairs of random variables (A, B), where $A \sim N(27.8, 7.5)$, $B \sim N(94.6, 19.6)$ and they are correlated $\rho_{A,B} = 0.92$. (These are summary statistics for BMI and weight circumference from NHANES 2017-2018.) Plot the marginal distributions of each variable and a scatterplot of their joint distribution.

10. Bootstrapping

Download the dataset "data_NHANES_bp.csv" from Canvas, which has blood pressure data (mean systolic) for 6,077 individuals from the National Health and Examination Nutrition Survey (NHANES) 2017-2018.

a. Calculate the mean of the blood pressure distribution. Bootstrap the dataset 1,000 times and estimate the 95% CI around the point estimate for the mean. Ignore the sample weights and survey design variables.

Hint: Write a function `calcMean(double vals[])`.

b. Repeat part a, but now account for each individual's sample weight when calculating the mean. (Also account for sample weights when bootstrapping.)

Hint: Write a function `calcWtMean(double vals[], double wts[])`.

c. Repeat part a, but now account for each individual's PSU and stratum in addition to their sample weight.

Hint: There are 15 strata and 2 PSUs per stratum. You could use a matrix of ArrayLists to keep track of which individuals belong to each stratum/PSU: `ArrayList<Integer> indices[][]=new ArrayList[numStrata][numPSU];`